

Core Idea. We adopt a two-stage prompt optimization strategy that combines textual and numeric gradients to leverage their complementary strengths. Textual gradients enable large improvements in the early stages by making high-level semantic revisions to the prompt; however, as optimization approaches a near-optimal region, these updates tend to become unstable and may fluctuate. In contrast, numeric gradient-based optimization progresses more slowly but provides fine-grained guidance by identifying which specific tokens are most important to update. Therefore, we first apply textual gradient updates to quickly reach a strong initial solution, and then refine it using numeric gradients to achieve more stable and precise improvements near the optimum.

Overview of Algorithm 1: Prompt Optimization We are given an initial Prompt, a train set and a validation set. We will optimize the prompt by using textual gradient for many iterations using evolutionary search. At the end, we keep the top prompt from the population for the next stage (numeric gradient based optimization). We then further refine the prompt by using numerical gradients for several iterations also using an evolutionary search. Finally, we keep the top prompt from the population.

Overview of Algorithm 2: Textual Gradient-based Optimization We are given an initial Prompt, a train set and a validation set. The population consists of prompts, starting with the initial prompt. In each step, a prompt is sampled from the population. A small mini-batch is then sampled from the training set. The sampled prompt is used to generate predictions on this mini-batch. From these predictions, a feedback set is constructed by selecting a few correct and incorrect examples. The LLM then analyzes this feedback set and produces textual feedback describing why the prompt made correct or incorrect predictions. Finally, the prompt is revised using this feedback to produce an updated prompt, which is added back to the population.

Overview of Algorithm 3: Numeric Gradient-based Optimization We are given an initial prompt, a training set, and a validation set. The population is initialized with the initial prompt. In each iteration, a prompt is sampled from the population. A small subset of data is then sampled from the training set. Using this data, the algorithm computes token-level gradients for the sampled prompt. It then identifies the regions of the prompt with the highest gradients. For simplicity, each region is treated as a single token, and only the top three regions are considered. **Next, the algorithm generates candidate replacements for each of these regions. This can be done in two ways. 1) The first way is to ask an LLM to generate replacement candidates by marking the target region with a <target> span. 2) The second approach uses the language modeling probabilities to predict the next token given the prompt prefix (i.e., the text before the region) and selects topK replacements (tokens) based on these probabilities.** After candidate replacements are obtained for each region, the algorithm combines them to create new prompt variants. The new prompts are then scored using their loss and perplexity on the sampled training data. The few top prompts with the best (lowest) scores are kept. Finally, these prompts are evaluated on the validation set and added to the population.

*Line 17 in Algorithm 3: *Instead of just the prefix, we prepend the original prompt as a prefix during autoregression:* We prepend the original prompt prefix to provide context about the full prompt during autoregressive generation. Using only the prefix of the prompt provides insufficient information about the task and ignores the right-side context of the prompt, leading the LLM to generate unrelated or inconsistent tokens for replacements. Including this context ensures that candidate tokens remain aligned with the region and the overall prompt structure.

Algorithm 1 Prompt Optimization

Input: Initial prompt P , training set $\mathcal{D}_{\text{train}}$, validation set $\mathcal{D}_{\text{val}}$

Output: Optimized prompt P^*

- 1: $P^* \leftarrow \text{TextualGradientUpdate}(P, \mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, T)$
 - 2: $P^* \leftarrow \text{NumericGradientUpdate}(P^*, \mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, Q)$
 - 3: **return** P^*
-

- ▷ We run Textual Gradient based optimization for T iterations
- ▷ We run Numeric Gradient based update for Q iterations

Algorithm 2 Textual Gradient Update

Input: Initial prompt P , training set $\mathcal{D}_{\text{train}}$, validation set $\mathcal{D}_{\text{val}}$, number of iterations T

Output: Optimized prompt P^*

```
1: Initialize  $\mathcal{P}_{\text{text}} \leftarrow \{P\}$ 
2: for  $t = 1$  to  $T$  do                                     ▷ will run  $T$  iterations for optimization
3:   Sample a prompt  $P_t \sim \mathcal{P}_{\text{text}}$ 
4:   Sample a mini-batch  $\mathcal{B} \subset \mathcal{D}_{\text{train}}$ 
5:   Use an LLM to infer predictions for  $\mathcal{B}$  under  $P_t$ 
6:   Construct a feedback set  $\mathcal{D}_{\text{feedback}}$  with three predictions (mix of corrects and incorrects)
7:   Use an LLM to analyze the predictions and produce feedbacks  $R$       ▷ Each feedbacks are “why the model made correct/incorrect
   prediction”
8:   Revise  $P_t$  using  $R$  to obtain  $P'_t$ 
9:   Compute  $\text{Score}(P'_t, \mathcal{D}_{\text{val}})$ 
10:  Add  $P'_t$  to  $\mathcal{P}_{\text{text}}$ 
11: end for
12:  $P^* \leftarrow \arg \max_{P \in \mathcal{P}_{\text{text}}} \text{Score}(P, \mathcal{D}_{\text{val}})$ 
13: return  $P^*$ 
```

Algorithm 3 Numeric Gradient Update

Input: Initial prompt P^* , training set $\mathcal{D}_{\text{train}}$, validation set $\mathcal{D}_{\text{val}}$, number of iterations Q

Output: Optimized prompt P^*

```
1: Initialize  $\mathcal{P}_{\text{num}} \leftarrow \{P^*\}$ 
2: for  $q = 1$  to  $Q$  do ▷ will run  $Q$  iterations for optimization
3:   Sample a prompt  $P_q \sim \mathcal{P}_{\text{num}}$ 
4:   Sample a mini-batch from  $\mathcal{D}_{\text{train}}$  to construct a feedback set  $\mathcal{D}_{\text{grad}}$ 
5:   Run inference using  $P_q$  on  $\mathcal{D}_{\text{grad}}$  and compute token-level gradients
6:   Select the top three gradient regions and store them in  $\mathcal{R}$  ▷ each region is comprised of one token for now
7:   Initialize an empty candidate map  $\mathcal{C}$  ▷ Here, for each region, we will store the candidates
8:   if  $m = \text{GENERATE\_CANDIDATES\_BY\_PROMPTING}$  then ▷ In this method, we use LLM to generate candidates via prompting
9:     for all  $r \in \mathcal{R}$  do ▷ iterate through each region
10:       $\tilde{P} =$  updated prompt  $P_q$  where region  $r$  is marked using a <target> span ▷ we mark the region by <target> span
11:      Generate candidate replacements  $C_r$  on the prompt  $\tilde{P}$  ▷ we use LLM to generate few candidates that can replace the region
12:      Store  $C_r$  in  $\mathcal{C}$  for region  $r$  ▷ store the candidates in a map for region  $r$ 
13:     end for
14:   else if  $m = \text{GENERATE\_CANDIDATES\_BY\_AUTOREGRESSION}$  then ▷ Here, we use LM probabilities to generate candidates (tokens)
15:     for all  $r \in \mathcal{R}$  do ▷ iterate through each region
16:       Let  $i$  be the index of region  $r$  in  $P_q$ 
17:        $\tilde{P} = P_q[0 : (i - 1)]$  ▷ our goal is to predict tokens for position  $i$ ; so, we take the tokens upto  $\leq (i - 1)$  position as input
18:       Candidate tokens,  $C_r = \arg \text{topK}_{t \in V} \text{LLM}(t \mid \tilde{P})$  ▷ given  $\tilde{P}$ , we collect topK tokens that has highest LM probabilities
19:       Store  $C_r$  in  $\mathcal{C}$  for region  $r$  ▷ store the candidates in a map for region  $r$ 
20:     end for
21:   end if
22:   Generate new prompts  $\mathcal{P}_{\text{cand}}$  by combining candidates  $C_r$  across regions
23:   Initialize an empty score map  $\mathcal{S}$  ▷ Here, for new prompt in  $\mathcal{P}_{\text{cand}}$ , we will store the scores (loss + perplexity)
24:   for all  $P \in \mathcal{P}_{\text{cand}}$  do
25:     Compute  $s = \text{Loss}_{\text{cross\_entropy}}(P, \mathcal{D}_{\text{grad}}) + \lambda \text{Perplexity}(P)$  ▷ calculate the score of the prompt
26:     Store score  $s$  in  $\mathcal{S}$  for prompt  $P$ 
27:   end for
28:   Select the top- $K$  prompts  $\mathcal{P}'$  from  $\mathcal{P}_{\text{cand}}$  according to  $\mathcal{S}$ 
29:   Add  $\mathcal{P}'$  to  $\mathcal{P}_{\text{num}}$  with validation scores on  $\mathcal{D}_{\text{val}}$ 
30: end for
31:  $P^* \leftarrow \arg \max_{P \in \mathcal{P}_{\text{num}}} \text{Score}(P, \mathcal{D}_{\text{val}})$ 
32: return  $P^*$ 
```
